

PCAP LAB-3

Pragun Kakkar – 240968332

Q1) Write a parallel program using OpenMP to implement the Selection sort algorithm. Compute the efficiency and plot the speed up for varying input size and thread number.
Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

#define NUM_SIZES 5
#define NUM_THREAD_COUNTS 5

/* -----
   Serial Selection Sort
   ----- */
void selection_sort_serial(int a[], int n)
{
    int i, j, min_idx, temp;

    for (i = 0; i < n - 1; i++)
    {
        min_idx = i;

        for (j = i + 1; j < n; j++)
        {
            if (a[j] < a[min_idx])
                min_idx = j;
        }

        temp = a[i];
        a[i] = a[min_idx];
        a[min_idx] = temp;
    }
}

/* -----
   Parallel Selection Sort using OpenMP
   ----- */
void selection_sort_parallel(int a[], int n, int num_threads)
{
    int i;

    omp_set_num_threads(num_threads);

    for (i = 0; i < n - 1; i++)
    {
        int min_idx = i;
        int min_value = a[i];
```

```

/*
 * Each thread finds its local minimum.
 */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int total_threads = omp_get_num_threads();

    int local_min_idx = i;
    int local_min_value = a[i];

    /*
     * IMPORTANT:
     * j is declared INSIDE the parallel block,
     * therefore every thread has its own j.
     */
    int j;

    /*
     * Divide the remaining array among threads.
     */
    int remaining = n - (i + 1);
    int chunk = (remaining + total_threads - 1)
                / total_threads;

    int start = i + 1 + tid * chunk;
    int end = start + chunk;

    if (end > n)
        end = n;

    if (start < n)
    {
        for (j = start; j < end; j++)
        {
            if (a[j] < local_min_value)
            {
                local_min_value = a[j];
                local_min_idx = j;
            }
        }
    }

    /*
     * Safely compare each thread's local minimum
     * with the global minimum.
     */
    #pragma omp critical
    {
        if (local_min_value < min_value)
        {

```

```

        min_value = local_min_value;
        min_idx = local_min_idx;
    }
}

/*
 * Swap the global minimum with the current position.
 */
if (min_idx != i)
{
    int temp = a[i];
    a[i] = a[min_idx];
    a[min_idx] = temp;
}
}
}

```

```

/* -----
Check whether array is sorted
----- */

```

```

int is_sorted(int a[], int n)

```

```

{
    int i;

    for (i = 0; i < n - 1; i++)
    {
        if (a[i] > a[i + 1])
            return 0;
    }

```

```

    return 1;
}

```

```

/* -----
Generate random array
----- */

```

```

void generate_array(int a[], int n)

```

```

{
    int i;

    for (i = 0; i < n; i++)
        a[i] = rand() % 100000;
}

```

```

/* -----
Main
----- */

```

```

int main()

```

```

{
    int sizes[NUM_SIZES] = {
        1000,

```

```

    2000,
    4000,
    8000,
    12000
};

int thread_counts[NUM_THREAD_COUNTS] = {
    1, 2, 4, 8, 16
};

FILE *fp;

printf("Pragun Kakkar - 240968332\n");

printf("\nParallel Selection Sort using OpenMP\n");
printf("=====\n");

printf("\n%-10s %-8s %-15s %-15s %-12s %-15s\n",
    "Size",
    "Threads",
    "Serial(s)",
    "Parallel(s)",
    "Speedup",
    "Efficiency(%)");

printf("-----\n");

fp = fopen("selection_sort_results.csv", "w");

if (fp == NULL)
{
    printf("Error opening CSV file.\n");
    return 1;
}

fprintf(fp,
    "InputSize,Threads,SerialTime,ParallelTime,Speedup,Efficiency\n");

srand(42);

for (int s = 0; s < NUM_SIZES; s++)
{
    int n = sizes[s];

    int *original = malloc(n * sizeof(int));
    int *serial_array = malloc(n * sizeof(int));
    int *parallel_array = malloc(n * sizeof(int));

    if (original == NULL ||
        serial_array == NULL ||
        parallel_array == NULL)
    {

```

```

    printf("Memory allocation failed.\n");
    return 1;
}

generate_array(original, n);

/*
 * -----
 * SERIAL SORT
 * -----
 */
memcpy(serial_array,
        original,
        n * sizeof(int));

double start = omp_get_wtime();

selection_sort_serial(serial_array, n);

double end = omp_get_wtime();

double serial_time = end - start;

if (!is_sorted(serial_array, n))
{
    printf("Serial sorting failed!\n");
    return 1;
}

/*
 * -----
 * PARALLEL SORT
 * -----
 */
for (int t = 0; t < NUM_THREAD_COUNTS; t++)
{
    int threads = thread_counts[t];

    memcpy(parallel_array,
            original,
            n * sizeof(int));

    start = omp_get_wtime();

    selection_sort_parallel(
        parallel_array,
        n,
        threads
    );

    end = omp_get_wtime();

```

```

double parallel_time = end - start;

if (!is_sorted(parallel_array, n))
{
    printf("Parallel sorting failed for %d threads!\n",
        threads);

    free(original);
    free(serial_array);
    free(parallel_array);
    fclose(fp);

    return 1;
}

double speedup =
    serial_time / parallel_time;

double efficiency =
    (speedup / threads) * 100.0;

printf("%-10d %-8d %-15.6f %-15.6f %-12.3f %-15.2f\n",
    n,
    threads,
    serial_time,
    parallel_time,
    speedup,
    efficiency);

fprintf(fp,
    "%d,%d,%.8f,%.8f,%.6f,%.4f\n",
    n,
    threads,
    serial_time,
    parallel_time,
    speedup,
    efficiency);
}

free(original);
free(serial_array);
free(parallel_array);
}

fclose(fp);

printf("\nResults saved to selection_sort_results.csv\n");

return 0;
}

```

```

• (base) mca@computnglab25-14:~/Documents/240968332/lab3$ gcc question1.c -fopenmp -o q1
• (base) mca@computnglab25-14:~/Documents/240968332/lab3$ ./q1
Pragun Kakkar - 240968332

```

Parallel Selection Sort using OpenMP

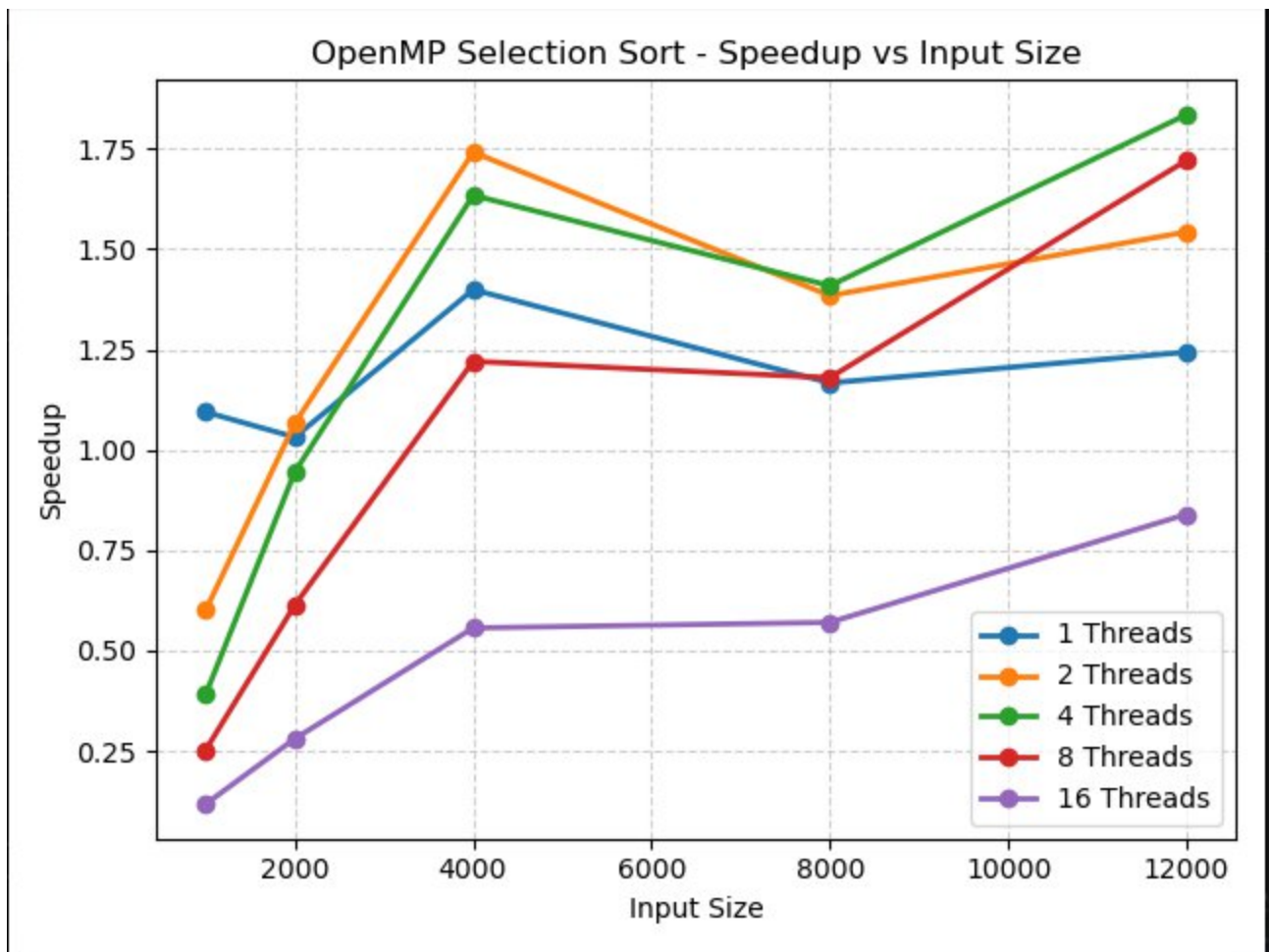
=====

Size	Threads	Serial(s)	Parallel(s)	Speedup	Efficiency(%)
1000	1	0.000648	0.000765	0.846	84.64
1000	2	0.000648	0.001063	0.609	30.47
1000	4	0.000648	0.001310	0.495	12.36
1000	8	0.000648	0.002225	0.291	3.64
1000	16	0.000648	0.004680	0.138	0.87
2000	1	0.002521	0.002418	1.042	104.25
2000	2	0.002521	0.002422	1.041	52.03
2000	4	0.002521	0.002514	1.003	25.07
2000	8	0.002521	0.004834	0.521	6.52
2000	16	0.002521	0.008389	0.300	1.88
4000	1	0.007073	0.006396	1.106	110.58
4000	2	0.007073	0.004680	1.511	75.56
4000	4	0.007073	0.005492	1.288	32.20
4000	8	0.007073	0.008278	0.854	10.68
4000	16	0.007073	0.017156	0.412	2.58
8000	1	0.025148	0.017360	1.449	144.86
8000	2	0.025148	0.012812	1.963	98.14
8000	4	0.025148	0.013933	1.805	45.12
8000	8	0.025148	0.016797	1.497	18.71
8000	16	0.025148	0.035214	0.714	4.46
12000	1	0.050126	0.036027	1.391	139.14
12000	2	0.050126	0.027697	1.810	90.49
12000	4	0.050126	0.024223	2.069	51.73
12000	8	0.050126	0.026392	1.899	23.74
12000	16	0.050126	0.052515	0.955	5.97

Results saved to selection_sort_results.csv

```
❖ (base) mca@computnglab25-14:~/Documents/240968332/lab3$ █
```

Output:



Graph:

Q2) Write a parallel program using OpenMP to implement sequential search algorithm. Compute the efficiency and plot the speed up for varying input size and thread number. Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define NUM_SIZES 5
#define NUM_THREAD_COUNTS 5

/* -----
   Sequential Search - Serial Version
   ----- */
int sequential_search_serial(int a[], int n, int key)
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (a[i] == key)
            return i;
    }

    return -1;
}

/* -----
   Sequential Search - Parallel Version

   The array is divided among OpenMP threads.
   Each thread searches its assigned portion.

   reduction(min:result) ensures that the smallest
   index found is returned.
   ----- */
int sequential_search_parallel(int a[], int n, int key, int num_threads)
{
    int result = n;

    omp_set_num_threads(num_threads);

    #pragma omp parallel for schedule(static) reduction(min:result)
    for (int i = 0; i < n; i++)
    {
        if (a[i] == key)
        {
            if (i < result)
                result = i;
        }
    }
}
```

```

        if (result == n)
            return -1;

    return result;
}

/* -----
   Generate array
   ----- */
void generate_array(int a[], int n)
{
    for (int i = 0; i < n; i++)
    {
        a[i] = i;
    }
}

/* -----
   Main
   ----- */
int main()
{
    int sizes[NUM_SIZES] = {
        100000,
        500000,
        1000000,
        5000000,
        10000000
    };

    int thread_counts[NUM_THREAD_COUNTS] = {
        1, 2, 4, 8, 16
    };

    printf("Pragun Kakkar - 240968332\n");

    printf("\nParallel Sequential Search using OpenMP\n");
    printf("=====\n");

    printf("\n%-12s %-8s %-15s %-15s %-12s %-15s\n",
        "Input Size",
        "Threads",
        "Serial(s)",
        "Parallel(s)",
        "Speedup",
        "Efficiency(%)");

    printf("-----\n");

    FILE *fp = fopen("sequential_search_results.csv", "w");

    if (fp == NULL)

```

```

{
    printf("Error opening CSV file.\n");
    return 1;
}

fprintf(fp,
        "InputSize,Threads,SerialTime,ParallelTime,Speedup,Efficiency\n");

for (int s = 0; s < NUM_SIZES; s++)
{
    int n = sizes[s];

    int *array = (int *)malloc(n * sizeof(int));

    if (array == NULL)
    {
        printf("Memory allocation failed for size %d.\n", n);
        fclose(fp);
        return 1;
    }

    /*
     * Create sorted array:
     *
     * 0, 1, 2, 3, ... n-1
     */
    generate_array(array, n);

    /*
     * Search for the LAST element.
     *
     * This forces the entire array to be searched
     * in the serial version.
     */
    int key = n - 1;

    /* -----
     Serial Search
     ----- */
    double start = omp_get_wtime();

    int serial_result =
        sequential_search_serial(array, n, key);

    double end = omp_get_wtime();

    double serial_time = end - start;

    /*
     * Verify serial result
     */
    if (serial_result != key)

```

```

{
    printf("Serial search failed!\n");
    free(array);
    fclose(fp);
    return 1;
}

/* -----
Parallel Search
----- */
for (int t = 0; t < NUM_THREAD_COUNTS; t++)
{
    int threads = thread_counts[t];

    start = omp_get_wtime();

    int parallel_result =
        sequential_search_parallel(
            array,
            n,
            key,
            threads
        );

    end = omp_get_wtime();

    double parallel_time = end - start;

    /*
    * Verify parallel result
    */
    if (parallel_result != key)
    {
        printf("Parallel search failed for %d threads!\n",
            threads);

        free(array);
        fclose(fp);
        return 1;
    }

    /*
    * Calculate speedup
    *
    * Speedup = Serial Time / Parallel Time
    */
    double speedup =
        serial_time / parallel_time;

    /*
    * Calculate efficiency
    *

```

```

    * Efficiency = Speedup / Number of Threads
    *           × 100
    */
double efficiency =
    (speedup / threads) * 100.0;

printf("%-12d %-8d %-15.6f %-15.6f %-12.3f %-15.2f\n",
    n,
    threads,
    serial_time,
    parallel_time,
    speedup,
    efficiency);

fprintf(fp,
    "%d,%d,%.8f,%.8f,%.6f,%.4f\n",
    n,
    threads,
    serial_time,
    parallel_time,
    speedup,
    efficiency);
}

free(array);
}

fclose(fp);

printf("\nResults saved to: sequential_search_results.csv\n");

return 0;
}

```

```

• (base) mca@computnglab25-14:~/Documents/240968332/lab3$ gcc question2.c -fopenmp -o q2
• (base) mca@computnglab25-14:~/Documents/240968332/lab3$ ./q2
Pragun Kakkar - 240968332

```

Parallel Sequential Search using OpenMP

=====

Input Size	Threads	Serial(s)	Parallel(s)	Speedup	Efficiency(%)
100000	1	0.000084	0.000082	1.021	102.15
100000	2	0.000084	0.000153	0.546	27.32
100000	4	0.000084	0.000112	0.746	18.64
100000	8	0.000084	0.000123	0.680	8.50
100000	16	0.000084	0.000220	0.381	2.38
500000	1	0.000421	0.000359	1.173	117.26
500000	2	0.000421	0.000190	2.212	110.61
500000	4	0.000421	0.000191	2.206	55.16
500000	8	0.000421	0.000090	4.705	58.81
500000	16	0.000421	0.000209	2.015	12.59
1000000	1	0.000838	0.000720	1.164	116.40
1000000	2	0.000838	0.000376	2.226	111.28
1000000	4	0.000838	0.000260	3.223	80.57
1000000	8	0.000838	0.000211	3.964	49.54
1000000	16	0.000838	0.000263	3.186	19.91
5000000	1	0.004363	0.003772	1.157	115.67
5000000	2	0.004363	0.003134	1.392	69.61
5000000	4	0.004363	0.000997	4.375	109.37
5000000	8	0.004363	0.000534	8.166	102.08
5000000	16	0.004363	0.000483	9.039	56.50
10000000	1	0.004902	0.005138	0.954	95.41
10000000	2	0.004902	0.003685	1.330	66.52
10000000	4	0.004902	0.002067	2.372	59.31
10000000	8	0.004902	0.001044	4.694	58.67
10000000	16	0.004902	0.000651	7.530	47.06

Results saved to: sequential_search_results.csv

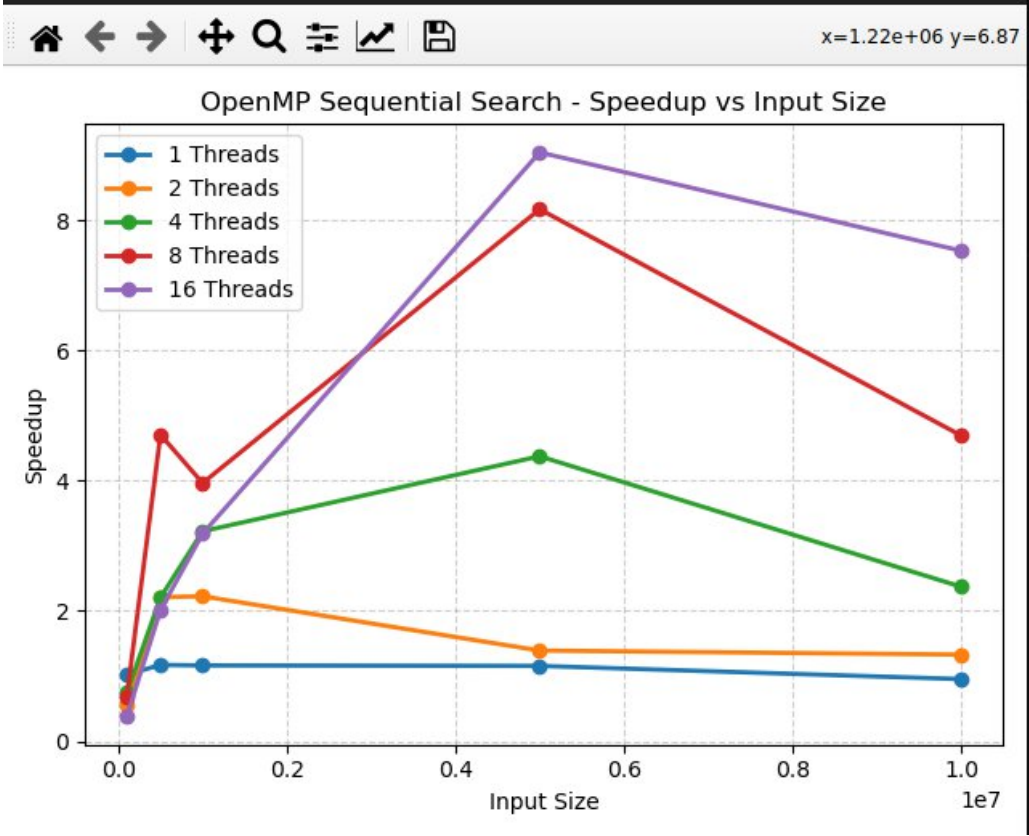
```

❖ (base) mca@computnglab25-14:~/Documents/240968332/lab3$ 

```

Output:

Graph:



Q3) Write a parallel program using OpenMP to perform vector addition, subtraction, multiplication. Demonstrate task level parallelism. Analyze the speedup and efficiency of the parallelized code.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define NUM_SIZES 5
#define NUM_THREAD_COUNTS 5

/* -----
  Serial Vector Operations
  ----- */

void vector_add_serial(double *A, double *B, double *C, int n)
{
    for (int i = 0; i < n; i++)
        C[i] = A[i] + B[i];
}

void vector_sub_serial(double *A, double *B, double *D, int n)
{
    for (int i = 0; i < n; i++)
        D[i] = A[i] - B[i];
}

void vector_mul_serial(double *A, double *B, double *E, int n)
{
    for (int i = 0; i < n; i++)
        E[i] = A[i] * B[i];
}

/* -----
  Parallel Vector Operations using TASK-LEVEL PARALLELISM

  Three independent tasks are created:

      Task 1 -> Vector Addition
      Task 2 -> Vector Subtraction
      Task 3 -> Vector Multiplication

  Since these operations are independent, different
  OpenMP threads can execute them simultaneously.
  ----- */

void vector_operations_parallel(
    double *A,
    double *B,
    double *C,
    double *D,
```

```

double *E,
int n,
int num_threads)
{
    omp_set_num_threads(num_threads);

    #pragma omp parallel
    {
        #pragma omp single
        {
            /*
             * TASK 1: Vector Addition
             */
            #pragma omp task
            {
                for (int i = 0; i < n; i++)
                    C[i] = A[i] + B[i];
            }

            /*
             * TASK 2: Vector Subtraction
             */
            #pragma omp task
            {
                for (int i = 0; i < n; i++)
                    D[i] = A[i] - B[i];
            }

            /*
             * TASK 3: Vector Multiplication
             */
            #pragma omp task
            {
                for (int i = 0; i < n; i++)
                    E[i] = A[i] * B[i];
            }

            /*
             * Wait until all three tasks finish.
             */
            #pragma omp taskwait
        }
    }
}

/* -----
Verify results
----- */

int verify_results(
    double *A,
    double *B,

```

```

double *C,
double *D,
double *E,
int n)
{
    for (int i = 0; i < n; i++)
    {
        if (C[i] != A[i] + B[i])
            return 0;

        if (D[i] != A[i] - B[i])
            return 0;

        if (E[i] != A[i] * B[i])
            return 0;
    }

    return 1;
}

/* -----
Initialize vectors
----- */

void initialize_vectors(double *A, double *B, int n)
{
    for (int i = 0; i < n; i++)
    {
        A[i] = (double)(i + 1);
        B[i] = (double)(2 * (i + 1));
    }
}

/* -----
Main
----- */

int main()
{
    /*
    * Different input sizes
    */
    int sizes[NUM_SIZES] = {
        1000000,
        5000000,
        10000000,
        20000000,
        50000000
    };

    /*
    * Number of threads to test.

```

```

*
* There are only 3 independent tasks, so maximum
* useful task-level parallelism is approximately
* 3 threads.
*/
int thread_counts[NUM_THREAD_COUNTS] = {
    1, 2, 3, 4, 8
};

FILE *fp;

/*
* REQUIRED FIRST OUTPUT LINE
*/
printf("Pragun Kakkar - 240968332\n");

printf("\nOpenMP Task-Level Parallelism");
printf("\nVector Addition, Subtraction and Multiplication");
printf("\n=====");

printf("\n%-12s %-8s %-15s %-15s %-12s %-15s\n",
    "Input Size",
    "Threads",
    "Serial(s)",
    "Parallel(s)",
    "Speedup",
    "Efficiency(%)");

printf("-----\n");

/*
* Open CSV file
*/
fp = fopen("vector_task_results.csv", "w");

if (fp == NULL)
{
    printf("Error creating CSV file.\n");
    return 1;
}

fprintf(fp,
    "InputSize,Threads,SerialTime,ParallelTime,Speedup,Efficiency\n");

/*
* Test each input size
*/
for (int s = 0; s < NUM_SIZES; s++)
{
    int n = sizes[s];

    printf("\nTesting input size: %d\n", n);

```

```

/*
 * Allocate vectors
 */
double *A = (double *)malloc(n * sizeof(double));
double *B = (double *)malloc(n * sizeof(double));

double *C_serial = (double *)malloc(n * sizeof(double));
double *D_serial = (double *)malloc(n * sizeof(double));
double *E_serial = (double *)malloc(n * sizeof(double));

double *C_parallel = (double *)malloc(n * sizeof(double));
double *D_parallel = (double *)malloc(n * sizeof(double));
double *E_parallel = (double *)malloc(n * sizeof(double));

if (A == NULL || B == NULL ||
    C_serial == NULL || D_serial == NULL ||
    E_serial == NULL ||
    C_parallel == NULL || D_parallel == NULL ||
    E_parallel == NULL)
{
    printf("Memory allocation failed!\n");

    fclose(fp);
    return 1;
}

/*
 * Initialize input vectors
 */
initialize_vectors(A, B, n);

/* =====
   SERIAL EXECUTION
   ===== */

double start = omp_get_wtime();

/*
 * All three operations execute sequentially.
 */
vector_add_serial(
    A, B, C_serial, n
);

vector_sub_serial(
    A, B, D_serial, n
);

vector_mul_serial(
    A, B, E_serial, n
);

```

```

double end = omp_get_wtime();

double serial_time = end - start;

/*
 * Verify serial result
 */
if (!verify_results(
    A, B,
    C_serial,
    D_serial,
    E_serial,
    n))
{
    printf("Serial result verification failed!\n");

    fclose(fp);
    return 1;
}

/* =====
PARALLEL EXECUTION
===== */

for (int t = 0; t < NUM_THREAD_COUNTS; t++)
{
    int threads = thread_counts[t];

    start = omp_get_wtime();

    /*
     * Three vector operations are submitted
     * as independent OpenMP tasks.
     */
    vector_operations_parallel(
        A,
        B,
        C_parallel,
        D_parallel,
        E_parallel,
        n,
        threads
    );

    end = omp_get_wtime();

    double parallel_time = end - start;

    /*
     * Verify parallel result
     */

```

```

if (!verify_results(
    A, B,
    C_parallel,
    D_parallel,
    E_parallel,
    n))
{
    printf(
        "Parallel result verification failed "
        "for %d threads!\n",
        threads
    );

    fclose(fp);
    return 1;
}

/*
 * Speedup
 *
 *  $S = T_{\text{serial}} / T_{\text{parallel}}$ 
 */
double speedup =
    serial_time / parallel_time;

/*
 * Efficiency
 *
 *  $E = \text{Speedup} / \text{Number of Threads} * 100$ 
 */
double efficiency =
    (speedup / threads) * 100.0;

printf(
    "%-12d %-8d %-15.6f %-15.6f %-12.3f %-15.2f\n",
    n,
    threads,
    serial_time,
    parallel_time,
    speedup,
    efficiency
);

/*
 * Save to CSV
 */
fprintf(
    fp,
    "%d,%d,%.8f,%.8f,%.6f,%.4f\n",
    n,
    threads,
    serial_time,

```

```
        parallel_time,  
        speedup,  
        efficiency  
    );  
}  
  
/*  
 * Free memory  
 */  
free(A);  
free(B);  
  
free(C_serial);  
free(D_serial);  
free(E_serial);  
  
free(C_parallel);  
free(D_parallel);  
free(E_parallel);  
}  
  
fclose(fp);  
  
printf("\nResults saved to: vector_task_results.csv\n");  
  
return 0;  
}
```

Output:

```
• (base) mca@computnglab25-14:~/Documents/240968332/lab3$ gcc question3.c -fopenmp -o q3
• (base) mca@computnglab25-14:~/Documents/240968332/lab3$ ./q3
Pragun Kakkar - 240968332
```

OpenMP Task-Level Parallelism
Vector Addition, Subtraction and Multiplication
=====

Input Size	Threads	Serial(s)	Parallel(s)	Speedup	Efficiency(%)
------------	---------	-----------	-------------	---------	---------------

Testing input size: 1000000

1000000	1	0.008991	0.008013	1.122	112.20
1000000	2	0.008991	0.002988	3.009	150.47
1000000	3	0.008991	0.002047	4.391	146.38
1000000	4	0.008991	0.001727	5.206	130.15
1000000	8	0.008991	0.002002	4.491	56.14

Testing input size: 5000000

5000000	1	0.041424	0.038432	1.078	107.79
5000000	2	0.041424	0.014711	2.816	140.79
5000000	3	0.041424	0.014769	2.805	93.49
5000000	4	0.041424	0.013319	3.110	77.75
5000000	8	0.041424	0.013605	3.045	38.06

Testing input size: 10000000

10000000	1	0.079060	0.076301	1.036	103.62
10000000	2	0.079060	0.032607	2.425	121.23
10000000	3	0.079060	0.031705	2.494	83.12
10000000	4	0.079060	0.029039	2.723	68.06
10000000	8	0.079060	0.029127	2.714	33.93

Testing input size: 20000000

20000000	1	0.158881	0.155609	1.021	102.10
20000000	2	0.158881	0.067992	2.337	116.84
20000000	3	0.158881	0.046961	3.383	112.78
20000000	4	0.158881	0.052210	3.043	76.08
20000000	8	0.158881	0.059039	2.691	33.64

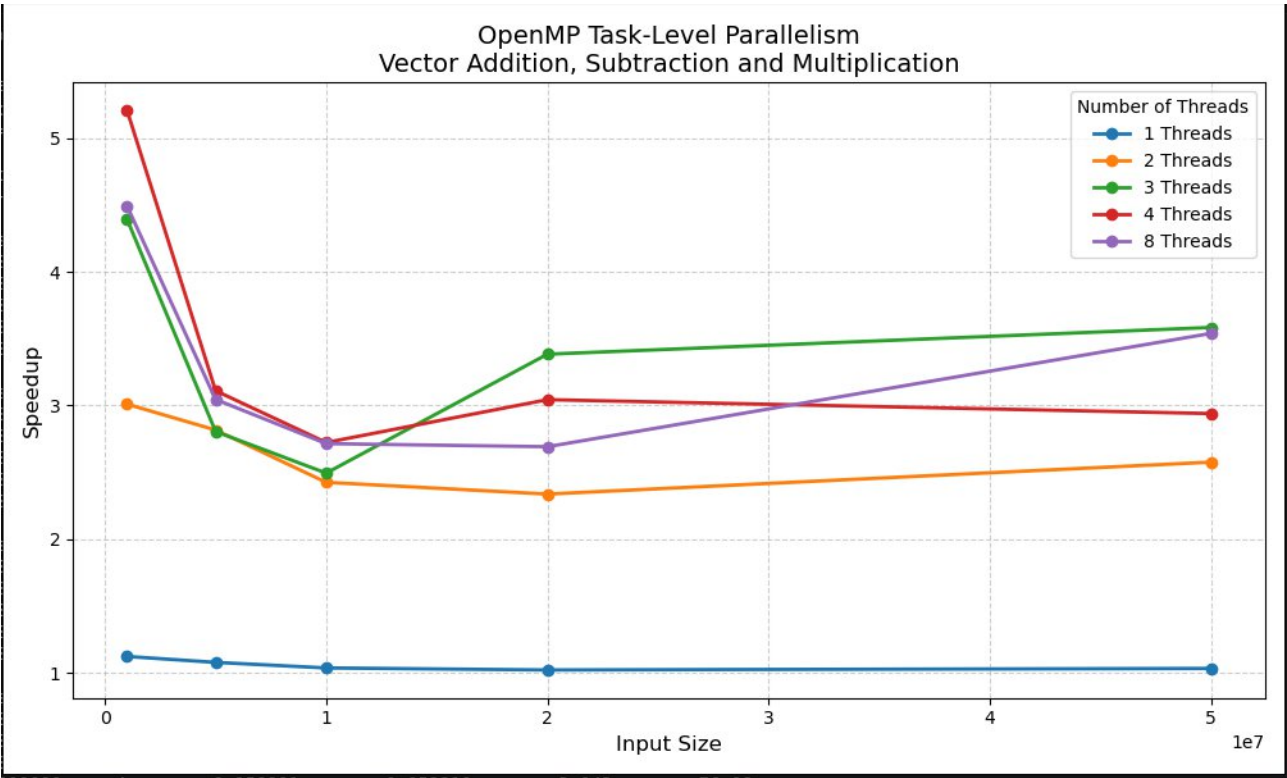
Testing input size: 50000000

50000000	1	0.394265	0.381913	1.032	103.23
50000000	2	0.394265	0.153076	2.576	128.78
50000000	3	0.394265	0.110034	3.583	119.44
50000000	4	0.394265	0.134146	2.939	73.48
50000000	8	0.394265	0.111410	3.539	44.24

Results saved to: vector_task_results.csv

```
❖ (base) mca@computnglab25-14:~/Documents/240968332/lab3$
```

Graph:



Q4) Write a parallel program using OpenMP to find sum of N numbers using the following constructs/clauses.

- Critical section
- Atomic
- Reduction
- Master
- Locks

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define NUM_SIZES 5
#define NUM_THREAD_COUNTS 4

/* -----
Serial Sum
----- */
long long sum_serial(long long *a, long long n)
{
    long long sum = 0;

    for (long long i = 0; i < n; i++)
        sum += a[i];

    return sum;
}

/* -----
Sum using CRITICAL
----- */
long long sum_critical(long long *a, long long n, int threads)
{
    long long sum = 0;

    omp_set_num_threads(threads);

    #pragma omp parallel
    {
        long long local_sum = 0;

        #pragma omp for
        for (long long i = 0; i < n; i++)
        {
            local_sum += a[i];
        }

        #pragma omp critical
        {
            sum += local_sum;
        }
    }
}
```

```

    }
}

return sum;
}

/* -----
Sum using ATOMIC
----- */
long long sum_atomic(long long *a, long long n, int threads)
{
    long long sum = 0;

    omp_set_num_threads(threads);

    #pragma omp parallel for
    for (long long i = 0; i < n; i++)
    {
        #pragma omp atomic
        sum += a[i];
    }

    return sum;
}

/* -----
Sum using REDUCTION
----- */
long long sum_reduction(long long *a, long long n, int threads)
{
    long long sum = 0;

    omp_set_num_threads(threads);

    #pragma omp parallel for reduction(+:sum)
    for (long long i = 0; i < n; i++)
    {
        sum += a[i];
    }

    return sum;
}

/* -----
Sum using MASTER
----- */
long long sum_master(long long *a, long long n, int threads)
{
    long long sum = 0;

    long long *local_sum =
        (long long *)calloc(threads, sizeof(long long));

```

```

if (local_sum == NULL)
{
    printf("Memory allocation failed.\n");
    exit(1);
}

omp_set_num_threads(threads);

#pragma omp parallel
{
    int tid = omp_get_thread_num();

    /*
     * Each thread calculates its own partial sum.
     */
    #pragma omp for
    for (long long i = 0; i < n; i++)
    {
        local_sum[tid] += a[i];
    }

    /*
     * Only the master thread combines the
     * partial sums.
     */
    #pragma omp master
    {
        for (int i = 0; i < threads; i++)
        {
            sum += local_sum[i];
        }
    }
}

free(local_sum);

return sum;
}

/* -----
Sum using LOCKS
----- */
long long sum_locks(long long *a, long long n, int threads)
{
    long long sum = 0;

    omp_lock_t lock;

    omp_init_lock(&lock);

    omp_set_num_threads(threads);

```

```

#pragma omp parallel for
for (long long i = 0; i < n; i++)
{
    omp_set_lock(&lock);

    sum += a[i];

    omp_unset_lock(&lock);
}

omp_destroy_lock(&lock);

return sum;
}

/* -----
Save result to CSV
----- */
void save_result(
    FILE *fp,
    const char *method,
    long long n,
    int threads,
    double serial_time,
    double parallel_time)
{
    double speedup =
        serial_time / parallel_time;

    double efficiency =
        (speedup / threads) * 100.0;

    fprintf(
        fp,
        "%s,%lld,%d,%.8f,%.8f,%.6f,%.4f\n",
        method,
        n,
        threads,
        serial_time,
        parallel_time,
        speedup,
        efficiency
    );
}

/* -----
Main
----- */
int main()
{
    /*

```

```

* Input sizes
*/
long long sizes[NUM_SIZES] = {
    1000000,
    5000000,
    10000000,
    20000000,
    50000000
};

/*
* ONLY 4 THREAD COUNTS
*/
int thread_counts[NUM_THREAD_COUNTS] = {
    1, 2, 3, 4
};

FILE *fp;

/*
* REQUIRED FIRST OUTPUT LINE
*/
printf("Pragun Kakkar - 240968332\n");

printf("\nOpenMP Sum of N Numbers\n");
printf("=====\n");

printf("Constructs: Critical, Atomic, Reduction, Master, Locks\n");
printf("Threads tested: 1, 2, 3, 4\n");

/*
* Open CSV file
*/
fp = fopen("sum_openmp_results.csv", "w");

if (fp == NULL)
{
    printf("Error creating CSV file.\n");
    return 1;
}

fprintf(
    fp,
    "Method,InputSize,Threads,SerialTime,ParallelTime,Speedup,Efficiency\n"
);

/*
* =====
* LOOP THROUGH INPUT SIZES
* =====
*/
for (int s = 0; s < NUM_SIZES; s++)

```

```

{
    long long n = sizes[s];

    /*
     * Allocate array
     */
    long long *a =
        (long long *)malloc(n * sizeof(long long));

    if (a == NULL)
    {
        printf("Memory allocation failed for %lld.\n", n);
        fclose(fp);
        return 1;
    }

    /*
     * Fill array with 1.
     *
     * Therefore:
     *
     * Sum = N
     */
    for (long long i = 0; i < n; i++)
    {
        a[i] = 1;
    }

    /*
     * =====
     * SERIAL EXECUTION
     * =====
     */
    double start = omp_get_wtime();

    long long expected =
        sum_serial(a, n);

    double end = omp_get_wtime();

    double serial_time = end - start;

    /*
     * Print only one line for each input size.
     */
    printf("\nInput Size = %lld | Expected Sum = %lld\n",
        n,
        expected);

    /*
     * =====
     * TEST ALL THREAD COUNTS

```

```

* =====
*/
for (int t = 0; t < NUM_THREAD_COUNTS; t++)
{
    int threads = thread_counts[t];

    long long result;

    double parallel_time;
    double speedup;
    double efficiency;

    /*
    * -----
    * CRITICAL
    * -----
    */
    start = omp_get_wtime();

    result =
        sum_critical(a, n, threads);

    end = omp_get_wtime();

    parallel_time = end - start;

    if (result != expected)
    {
        printf("ERROR: Critical result incorrect!\n");
        free(a);
        fclose(fp);
        return 1;
    }

    speedup = serial_time / parallel_time;
    efficiency =
        (speedup / threads) * 100.0;

    save_result(
        fp,
        "Critical",
        n,
        threads,
        serial_time,
        parallel_time
    );

    /*
    * -----
    * ATOMIC
    * -----
    */
}

```

```

start = omp_get_wtime();

result =
    sum_atomic(a, n, threads);

end = omp_get_wtime();

parallel_time = end - start;

if (result != expected)
{
    printf("ERROR: Atomic result incorrect!\n");
    free(a);
    fclose(fp);
    return 1;
}

speedup = serial_time / parallel_time;
efficiency =
    (speedup / threads) * 100.0;

save_result(
    fp,
    "Atomic",
    n,
    threads,
    serial_time,
    parallel_time
);

/*
 * -----
 * REDUCTION
 * -----
 */
start = omp_get_wtime();

result =
    sum_reduction(a, n, threads);

end = omp_get_wtime();

parallel_time = end - start;

if (result != expected)
{
    printf("ERROR: Reduction result incorrect!\n");
    free(a);
    fclose(fp);
    return 1;
}

```

```

speedup = serial_time / parallel_time;
efficiency =
    (speedup / threads) * 100.0;

save_result(
    fp,
    "Reduction",
    n,
    threads,
    serial_time,
    parallel_time
);

/*
 * -----
 * MASTER
 * -----
 */
start = omp_get_wtime();

result =
    sum_master(a, n, threads);

end = omp_get_wtime();

parallel_time = end - start;

if (result != expected)
{
    printf("ERROR: Master result incorrect!\n");
    free(a);
    fclose(fp);
    return 1;
}

speedup = serial_time / parallel_time;
efficiency =
    (speedup / threads) * 100.0;

save_result(
    fp,
    "Master",
    n,
    threads,
    serial_time,
    parallel_time
);

/*
 * -----
 * LOCKS
 * -----

```

```

    */
    start = omp_get_wtime();

    result =
        sum_locks(a, n, threads);

    end = omp_get_wtime();

    parallel_time = end - start;

    if (result != expected)
    {
        printf("ERROR: Locks result incorrect!\n");
        free(a);
        fclose(fp);
        return 1;
    }

    speedup = serial_time / parallel_time;
    efficiency =
        (speedup / threads) * 100.0;

    save_result(
        fp,
        "Locks",
        n,
        threads,
        serial_time,
        parallel_time
    );
}

/*
 * Free array
 */
free(a);
}

/*
 * Close CSV
 */
fclose(fp);

printf("\n-----\n");
printf("All tests completed successfully.\n");
printf("Results saved to: sum_openmp_results.csv\n");
printf("-----\n");

return 0;
}

```

Ouput:

```
• (base) mca@computnlab25-14:~/Documents/240968332/lab3$ gcc question4.c -fopenmp -o q4
• (base) mca@computnlab25-14:~/Documents/240968332/lab3$ ./q4
Pragun Kakkar - 240968332

OpenMP Sum of N Numbers
=====
Constructs: Critical, Atomic, Reduction, Master, Locks
Threads tested: 1, 2, 3, 4

Input Size = 1000000 | Expected Sum = 1000000
Input Size = 5000000 | Expected Sum = 5000000
Input Size = 10000000 | Expected Sum = 10000000
Input Size = 20000000 | Expected Sum = 20000000
Input Size = 50000000 | Expected Sum = 50000000

-----
All tests completed successfully.
Results saved to: sum_openmp_results.csv
-----
(base) mca@computnlab25-14:~/Documents/240968332/lab3$
```

Q5) Write an OpenMP program to find the Summation of integers from a given interval. Analyze the performance of various iteration scheduling strategies.
Code;

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define NUM_THREADS 4
#define NUM_INTERVALS 5

/* -----
   Serial Summation
   ----- */
long long serial_sum(long long start, long long end)
{
    long long sum = 0;

    for (long long i = start; i <= end; i++)
    {
        sum += i;
    }

    return sum;
}

/* -----
   Parallel Summation using STATIC scheduling
   ----- */
long long parallel_static(
    long long start,
    long long end,
    int threads)
{
    long long sum = 0;

    omp_set_num_threads(threads);

    #pragma omp parallel for schedule(static) reduction(+:sum)
    for (long long i = start; i <= end; i++)
    {
        sum += i;
    }

    return sum;
}

/* -----
   Parallel Summation using DYNAMIC scheduling
   ----- */
long long parallel_dynamic(
    long long start,
```

```

    long long end,
    int threads)
{
    long long sum = 0;

    omp_set_num_threads(threads);

    #pragma omp parallel for schedule(dynamic, 1000) reduction(+:sum)
    for (long long i = start; i <= end; i++)
    {
        sum += i;
    }

    return sum;
}

```

```

/* -----
Parallel Summation using GUIDED scheduling
----- */

```

```

long long parallel_guided(
    long long start,
    long long end,
    int threads)
{
    long long sum = 0;

    omp_set_num_threads(threads);

    #pragma omp parallel for schedule(guided, 1000) reduction(+:sum)
    for (long long i = start; i <= end; i++)
    {
        sum += i;
    }

    return sum;
}

```

```

/* -----
Parallel Summation using AUTO scheduling
----- */

```

```

long long parallel_auto(
    long long start,
    long long end,
    int threads)
{
    long long sum = 0;

    omp_set_num_threads(threads);

    #pragma omp parallel for schedule(auto) reduction(+:sum)
    for (long long i = start; i <= end; i++)
    {

```

```

        sum += i;
    }

    return sum;
}

/* -----
Save results to CSV
----- */
void save_result(
    FILE *fp,
    const char *schedule,
    long long interval,
    int threads,
    double serial_time,
    double parallel_time)
{
    double speedup =
        serial_time / parallel_time;

    double efficiency =
        (speedup / threads) * 100.0;

    fprintf(
        fp,
        "%s,%lld,%d,%.8f,%.8f,%.6f,%.4f\n",
        schedule,
        interval,
        threads,
        serial_time,
        parallel_time,
        speedup,
        efficiency
    );
}

/* -----
Main
----- */
int main()
{
    /*
    * Different interval sizes.
    *
    * The interval is:
    *
    * 1 to interval
    */
    long long intervals[NUM_INTERVALS] = {
        1000000,
        5000000,
        10000000,

```

```

    200000000,
    500000000
};

/*
 * Number of threads.
 */
int thread_counts[NUM_THREADS] = {
    1, 2, 3, 4
};

FILE *fp;

/*
 * REQUIRED FIRST OUTPUT LINE
 */
printf("Pragun Kakkar - 240968332\n");

printf("\nOpenMP Summation of Integers over an Interval\n");
printf("=====\n");

printf("Scheduling strategies: Static, Dynamic, Guided, Auto\n");
printf("Threads tested: 1, 2, 3, 4\n");

/*
 * Open CSV file.
 */
fp = fopen("schedule_results.csv", "w");

if (fp == NULL)
{
    printf("Error creating CSV file.\n");
    return 1;
}

fprintf(
    fp,
    "Schedule,Interval,Threads,SerialTime,ParallelTime,Speedup,Efficiency\n"
);

/*
 * =====
 * TEST DIFFERENT INTERVALS
 * =====
 */
for (int s = 0; s < NUM_INTERVALS; s++)
{
    long long end = intervals[s];

    long long start_value = 1;

    /*

```

```

* -----
* SERIAL EXECUTION
* -----
*/
double start_time = omp_get_wtime();

long long expected =
    serial_sum(start_value, end);

double end_time = omp_get_wtime();

double serial_time =
    end_time - start_time;

printf(
    "\nInterval: [%lld, %lld] | Sum = %lld\n",
    start_value,
    end,
    expected
);

/*
* =====
* THREAD COUNTS
* =====
*/
for (int t = 0; t < NUM_THREADS; t++)
{
    int threads = thread_counts[t];

    long long result;

    double parallel_time;

    /*
    * =====
    * STATIC
    * =====
    */
    start_time = omp_get_wtime();

    result =
        parallel_static(
            start_value,
            end,
            threads
        );

    end_time = omp_get_wtime();

    parallel_time =
        end_time - start_time;

```

```

if (result != expected)
{
    printf("ERROR: Static result incorrect!\n");

    fclose(fp);
    return 1;
}

save_result(
    fp,
    "Static",
    end,
    threads,
    serial_time,
    parallel_time
);

/*
 * =====
 * DYNAMIC
 * =====
 */
start_time = omp_get_wtime();

result =
    parallel_dynamic(
        start_value,
        end,
        threads
    );

end_time = omp_get_wtime();

parallel_time =
    end_time - start_time;

if (result != expected)
{
    printf("ERROR: Dynamic result incorrect!\n");

    fclose(fp);
    return 1;
}

save_result(
    fp,
    "Dynamic",
    end,
    threads,
    serial_time,
    parallel_time

```

```

);

/*
 * =====
 * GUIDED
 * =====
 */
start_time = omp_get_wtime();

result =
    parallel_guided(
        start_value,
        end,
        threads
    );

end_time = omp_get_wtime();

parallel_time =
    end_time - start_time;

if (result != expected)
{
    printf("ERROR: Guided result incorrect!\n");

    fclose(fp);
    return 1;
}

save_result(
    fp,
    "Guided",
    end,
    threads,
    serial_time,
    parallel_time
);

/*
 * =====
 * AUTO
 * =====
 */
start_time = omp_get_wtime();

result =
    parallel_auto(
        start_value,
        end,
        threads
    );

```

```

end_time = omp_get_wtime();

parallel_time =
    end_time - start_time;

if (result != expected)
{
    printf("ERROR: Auto result incorrect!\n");

    fclose(fp);
    return 1;
}

save_result(
    fp,
    "Auto",
    end,
    threads,
    serial_time,
    parallel_time
);
}
}

fclose(fp);

printf("\n-----\n");
printf("All tests completed successfully.\n");
printf("Results saved to: schedule_results.csv\n");
printf("-----\n");

return 0;
}

```

Output:

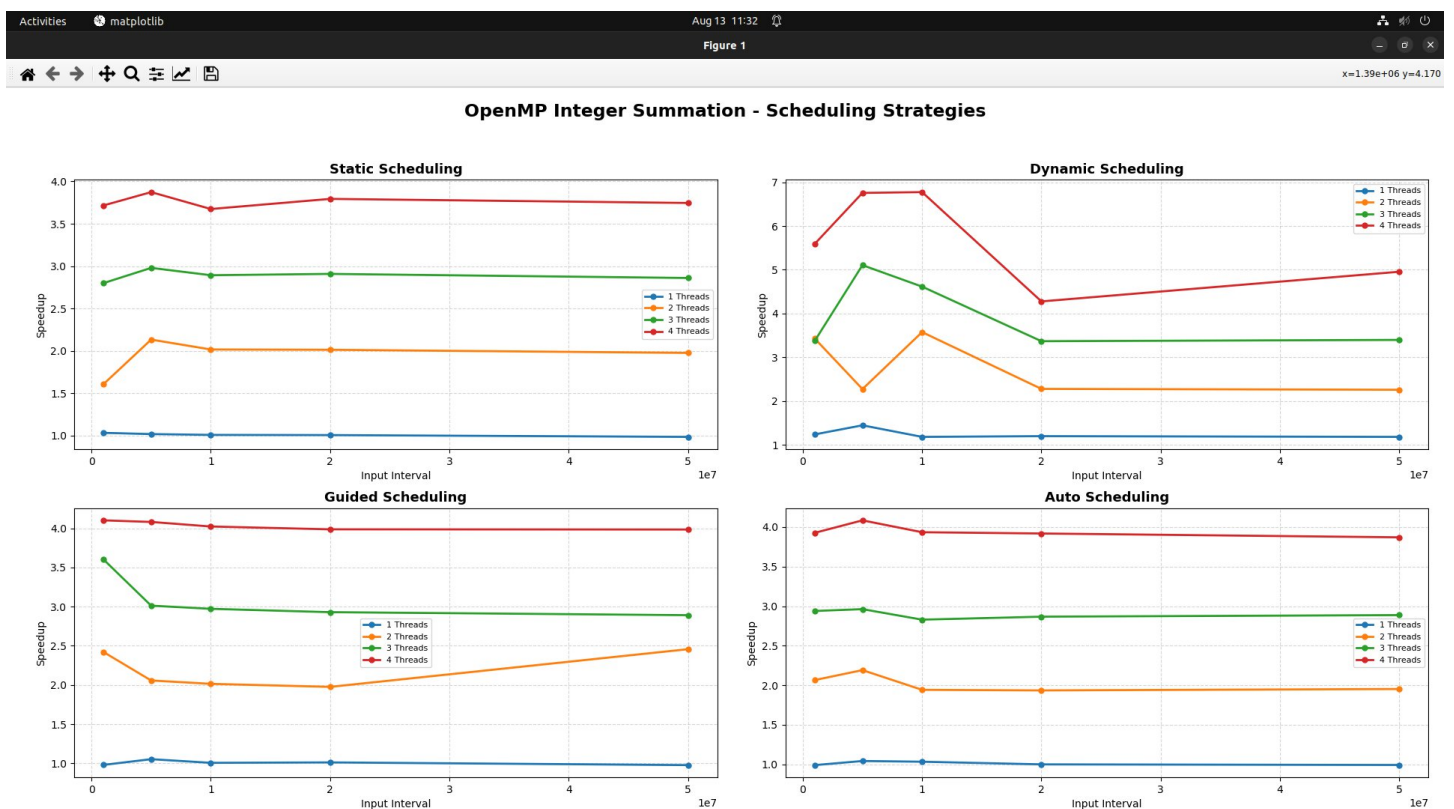
```
(base) mca@computnglab25-14:~/Documents/240968332/lab3$ gcc question5.c -fopenmp -o q5
(base) mca@computnglab25-14:~/Documents/240968332/lab3$ ./q5
Pragun Kakkar - 240968332

OpenMP Summation of Integers over an Interval
=====
Scheduling strategies: Static, Dynamic, Guided, Auto
Threads tested: 1, 2, 3, 4

Interval: [1, 1000000] | Sum = 500000500000
Interval: [1, 5000000] | Sum = 12500002500000
Interval: [1, 10000000] | Sum = 50000005000000
Interval: [1, 20000000] | Sum = 200000010000000
Interval: [1, 50000000] | Sum = 1250000025000000

-----
All tests completed successfully.
Results saved to: schedule_results.csv
-----
(base) mca@computnglab25-14:~/Documents/240968332/lab3$
```

Graph:



Write a parallel program using OpenMP to compute π using random shooting.

Hint: Shoot randomly into a square $[0, 1] \times [0, 1]$ and count how many shots hit inside the unit circle and how many do not. Then calculate the ratio of number of points lied inside the circle and total number of generated points. We know that, area of the circle is πr^2 and area of the square is $4r^2$. Now, for a very large number of generated points,

$$\pi = 4 * \frac{\text{No. of points generated inside the circle}}{\text{Number of points generated inside the square}}$$

Q6)

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define NUM_SIZES 5
#define NUM_THREADS 4

/* -----
Serial Monte Carlo computation of PI
----- */
long long serial_pi(long long n, unsigned int seed)
{
    long long inside = 0;

    unsigned int local_seed = seed;

    for (long long i = 0; i < n; i++)
    {
        double x =
            (double)rand_r(&local_seed) / RAND_MAX;

        double y =
            (double)rand_r(&local_seed) / RAND_MAX;

        /*
         * Check whether point lies inside
         * the quarter of the unit circle.
         */
        if ((x * x + y * y) <= 1.0)
        {
            inside++;
        }
    }

    return inside;
}
```

```

/* -----
Parallel Monte Carlo computation of PI
----- */
long long parallel_pi(
    long long n,
    int threads)
{
    long long inside = 0;

    omp_set_num_threads(threads);

    /*
     * Each thread has its own random seed.
     */
    #pragma omp parallel reduction(+:inside)
    {
        int tid = omp_get_thread_num();

        /*
         * Different seed for every thread.
         */
        unsigned int seed =
            (unsigned int)time(NULL)
            ^ (unsigned int)(tid * 1234567 + 987654);

        /*
         * Parallelize the generation of points.
         */
        #pragma omp for
        for (long long i = 0; i < n; i++)
        {
            double x =
                (double)rand_r(&seed) / RAND_MAX;

            double y =
                (double)rand_r(&seed) / RAND_MAX;

            /*
             * Point is inside quarter circle.
             */
            if ((x * x + y * y) <= 1.0)
            {
                inside++;
            }
        }
    }

    return inside;
}

/* -----

```

Main

```
----- */
int main()
{
    /*
     * Number of random points.
     */
    long long points[NUM_SIZES] = {
        1000000,
        5000000,
        10000000,
        20000000,
        50000000
    };

    /*
     * Threads to test.
     */
    int thread_counts[NUM_THREADS] = {
        1, 2, 3, 4
    };

    FILE *fp;

    /*
     * REQUIRED FIRST OUTPUT LINE
     */
    printf("Pragun Kakkar - 240968332\n");

    printf("\nMonte Carlo Computation of PI using OpenMP\n");
    printf("=====\n");

    printf("Threads tested: 1, 2, 3, 4\n");

    /*
     * Open CSV file.
     */
    fp = fopen("pi_results.csv", "w");

    if (fp == NULL)
    {
        printf("Error creating CSV file.\n");
        return 1;
    }

    fprintf(
        fp,
        "Points,Threads,SerialTime,ParallelTime,PI,Speedup,Efficiency\n"
    );

    /*
     * =====

```

```

* Test different numbers of points
* =====
*/
for (int s = 0; s < NUM_SIZES; s++)
{
    long long n = points[s];

    /*
    * -----
    * SERIAL EXECUTION
    * -----
    */

    double start = omp_get_wtime();

    long long serial_inside =
        serial_pi(
            n,
            (unsigned int)time(NULL)
        );

    double end = omp_get_wtime();

    double serial_time =
        end - start;

    /*
    * Calculate serial PI.
    */
    double serial_pi_value =
        4.0 * (double)serial_inside / (double)n;

    printf(
        "\nPoints = %lld | Serial PI = %.8f\n",
        n,
        serial_pi_value
    );

    /*
    * -----
    * PARALLEL EXECUTION
    * -----
    */
    for (int t = 0; t < NUM_THREADS; t++)
    {
        int threads = thread_counts[t];

        start = omp_get_wtime();

        long long parallel_inside =
            parallel_pi(
                n,

```

```

        threads
    );

end = omp_get_wtime();

double parallel_time =
    end - start;

/*
 * Calculate PI.
 */
double pi_value =
    4.0 *
    (double)parallel_inside /
    (double)n;

/*
 * Calculate speedup.
 */
double speedup =
    serial_time /
    parallel_time;

/*
 * Calculate efficiency.
 */
double efficiency =
    (speedup / threads) * 100.0;

/*
 * Display concise output.
 */
printf(
    "Threads = %d | PI = %.8f | Time = %.6f s | "
    "Speedup = %.3f | Efficiency = %.2f%%\n",
    threads,
    pi_value,
    parallel_time,
    speedup,
    efficiency
);

/*
 * Save results to CSV.
 */
fprintf(
    fp,
    "%lld,%d,%.8f,%.8f,%.8f,%.6f,%.4f\n",
    n,
    threads,
    serial_time,
    parallel_time,

```

```

        pi_value,
        speedup,
        efficiency
    );
}
}

fclose(fp);

printf("\n-----\n");
printf("Computation completed successfully.\n");
printf("Results saved to: pi_results.csv\n");
printf("-----\n");

return 0;
}

```

Output:

```

● (base) mca@computnglab25-14:~/Documents/240968332/lab3$ gcc question6.c -fopenmp -o q6
● (base) mca@computnglab25-14:~/Documents/240968332/lab3$ ./q6
Pragun Kakkar - 240968332

Monte Carlo Computation of PI using OpenMP
=====
Threads tested: 1, 2, 3, 4

Points = 1000000 | Serial PI = 3.14123600
Threads = 1 | PI = 3.13978000 | Time = 0.009739 s | Speedup = 1.082 | Efficiency = 108.21%
Threads = 2 | PI = 3.14183200 | Time = 0.004880 s | Speedup = 2.160 | Efficiency = 107.98%
Threads = 3 | PI = 3.14250000 | Time = 0.002912 s | Speedup = 3.619 | Efficiency = 120.64%
Threads = 4 | PI = 3.14216800 | Time = 0.002181 s | Speedup = 4.832 | Efficiency = 120.81%

Points = 5000000 | Serial PI = 3.14186640
Threads = 1 | PI = 3.14016480 | Time = 0.037858 s | Speedup = 1.022 | Efficiency = 102.15%
Threads = 2 | PI = 3.13994080 | Time = 0.019153 s | Speedup = 2.019 | Efficiency = 100.95%
Threads = 3 | PI = 3.14019600 | Time = 0.016687 s | Speedup = 2.318 | Efficiency = 77.25%
Threads = 4 | PI = 3.14091040 | Time = 0.010309 s | Speedup = 3.751 | Efficiency = 93.78%

Points = 10000000 | Serial PI = 3.14153800
Threads = 1 | PI = 3.14106840 | Time = 0.075686 s | Speedup = 1.011 | Efficiency = 101.13%
Threads = 2 | PI = 3.14089680 | Time = 0.037613 s | Speedup = 2.035 | Efficiency = 101.74%
Threads = 3 | PI = 3.14102000 | Time = 0.025536 s | Speedup = 2.997 | Efficiency = 99.91%
Threads = 4 | PI = 3.14143640 | Time = 0.020622 s | Speedup = 3.711 | Efficiency = 92.79%

Points = 20000000 | Serial PI = 3.14125580
Threads = 1 | PI = 3.14139880 | Time = 0.150610 s | Speedup = 1.014 | Efficiency = 101.44%
Threads = 2 | PI = 3.14095340 | Time = 0.075118 s | Speedup = 2.034 | Efficiency = 101.70%
Threads = 3 | PI = 3.14088320 | Time = 0.055275 s | Speedup = 2.764 | Efficiency = 92.14%
Threads = 4 | PI = 3.14106420 | Time = 0.038634 s | Speedup = 3.955 | Efficiency = 98.87%

Points = 50000000 | Serial PI = 3.14168056
Threads = 1 | PI = 3.14174808 | Time = 0.378802 s | Speedup = 1.010 | Efficiency = 100.96%
Threads = 2 | PI = 3.14142248 | Time = 0.190139 s | Speedup = 2.011 | Efficiency = 100.57%
Threads = 3 | PI = 3.14186200 | Time = 0.128662 s | Speedup = 2.973 | Efficiency = 99.09%
Threads = 4 | PI = 3.14177400 | Time = 0.095320 s | Speedup = 4.012 | Efficiency = 100.31%

-----
Computation completed successfully.
Results saved to: pi_results.csv
-----
❖ (base) mca@computnglab25-14:~/Documents/240968332/lab3$

```